大型代数方程组算法与数学

许进超, Penn State University

May 2018

Abstract

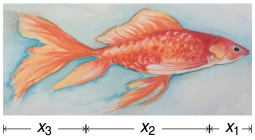
In this paper, 我们简要讨论 numerical methods for a very basic question in practice: namely the solution of linear algebraic systems of equations. From the oldest Gaussian elimination method to the moden multigrid methods, we demonstrate how mathematical techniques can be used to develop efficient methods for solve a practical problem. We will also discuss the comparison between the computer hardware development and mathematical algorithm developments.

1 引言

The topic we will discuss in this paper is mathematical technique for solving linear system of equations. Let us first give a very simple example from my childhood experience.

外公在我小时候给我出了一道算术题：鱼头 3 寸，鱼身是头尾之和，鱼尾是头身之半，鱼长几何？设鱼头、鱼身和鱼尾的长度分别是 x_1, x_2 和 x_3，根据题设条件，我们可以建立如下等量关系：

$$\begin{array}{rcrcrl} x_1 & & & & & = 3 \\ x_1 & - & x_2 & + & x_3 & = 0 \\ x_1 & + & x_2 & - & 2x_3 & = 0. \end{array}$$



通过求解上述线性系统，我们就可以求得鱼长为 $x_1 + x_2 + x_3 = 3 + 6 + 9 = 18$。用字母表示数，使得客观世界中的未知量可以用字母来表示，然后依据一些定律找出数量之间的等量关系，列成方程。如果这些方程关于未知量是线性的，我们称之为线性代数 是不是 规律 更好? 有 n 个未知量和 n 个线性代数方程的系统可以写成

$$\begin{cases} a_{11}x_1 + a_{12}x_2 & + \cdots + a_{1n}x_n = b_1, \\ a_{21}x_1 + a_{22}x_2 & + \cdots + a_{2n}x_n = b_2, \\ \quad\vdots & \\ a_{n1}x_1 + a_{n2}x_2 & + \cdots + a_{nn}x_n = b_n. \end{cases} \tag{1}$$

Most applications of mathematical applications to practical problems will amount to the solution of linear algebraic system as above. For example, 这样的线性代数方程组还可以来源于对微分方程的数值离散和对非线性代数方程组的线性逼近。求解线性方程组的数值算法是科学计算最基本的研究问题。高效的数值求解算法在现代工程学、物理学、化学、经济学和计算机学等科学研究中扮演着基础而重要的角色。

最古老也是最通用的求解线性方程组的方法是高斯消去法。该方法以数学家卡尔 · 高斯命名，它最早可见于成书于约公元前 150 年左右的中国古籍《九章算术》。高斯消去的核心想法是通过方程之间的相互变换、消解得到上三角或者下三角系统，然后再利用代入法完成求解。对于一

个 $n \times n$ 的矩阵, 高斯消元所需的浮点运算次数为 $\frac{2}{3}n^3 + n^2 + O(n)$, 对于非常巨大的 n, 这个算法因为过于费时而变得不可用。

因为代数方程求解以及高斯消除法的重要性，在高性能计算机领域，人们最为关注的测试对全球高级计算机系统进行排名的 TOP500 项目，就是通过高性能 LINPACK（HPL）采用高斯消元法求解 n 阶稠密线性代数方程组所花的时间来确定计算机系统的排名。如果测得系统计算时间为 T，那么该计算机系统的实际计算峰值为 $\frac{2n^3/3+2n^2}{T}$，测试结果以浮点计算每秒（FLOPs）给出。TOP500 项目从 1993 年运行开始，最强大的超级计算机一直在美国和日本之间角逐，中国在最近的十年内逐渐位列并领先于该榜单中。目前位列于前三的分别是中国国家超级计算中心（无锡）的神威 · 太湖之光、中国国家超级计算中心（广州）的天河二号和瑞士国家超级计算中心的 Piz Daint。神威 · 太湖之光的实测计算峰值性能为 93.015 PFLOPs，也就是实际每秒进行 93.015×10^{15} 次浮点运算。于是，如果求解 10^7 阶稠密线性代数方程组，就大概需要 2 小时；如果求解系统阶数为 10^8，就大概需要 83 天；如果求解系统阶数为 10^9，就大概需要 228 年。从这个角度来看，无论计算机性能如何提升，为了解决大规模计算问题，我们必须研究比高斯消元法更为高效的数值算法。

我们可以从两个角度来研发求解代数方程更为高效的数值算法。第一：我们在高斯消元法的基础上做改进。第二：我们用完全不同的方法，即迭代法。

2 高斯消去法的改进

如何提高高斯消元法的效率，是科学计算中大家一直关注的一个重要问题。我们在这里简单介绍两个途径：Strassen 方法以及稀疏矩阵方法。

Strassen 算法。 1969 年，Strassen 提出的利用分治思想结合特别的组合方式得到的快速矩阵相乘算法. 我们以 2×2 矩阵为例, 展示特殊的组合方式如何减少一次乘法运算. 考虑两个 2×2 矩阵的乘法:

$$\begin{pmatrix} r & s \\ t & u \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \cdot \begin{pmatrix} e & f \\ g & h \end{pmatrix}, \tag{2}$$

于是我们得到:

$$r = ae + bg, s = af + bh, t = ce + dh, u = cf + dg.$$

但是, 通过巧妙的组合, 我们可以得到:

$$r = P_5 + P_4 + P_6 - P_2, s = P_1 + P_2, t = P_3 + P_4, u = P_1 + P_5 - P_3 - P_7.$$

其中: $P_1 = a \cdot (f - h), P_2 = (a + b) \cdot h, P_3 = (c + d) \cdot e, P_4 = d \cdot (g - e), P_5 = (a + d) \cdot (e + h), P_6 = (b - d) \cdot (g + h), P_7 = (a - c) \cdot (e + f)$.

如果将 $a, b, \cdots, u$ 都想象成 $\frac{n}{2} \times \frac{n}{2}$ 的矩阵, 那么这里的乘法都是矩阵乘法, 于是我们得到分治思想, 并且在只考虑乘法的情况下, 我们得到 $n \times n$ 矩阵乘法运算量 $T(n)$ 的递归:

$$T(n) = 7T(n) + O(n^2). \tag{3}$$

从而, 我们将常规的矩阵与矩阵相乘 $\mathcal{O}(n^3)$ 的工作量降至到 $\mathcal{O}(n^{\log_2 7}) \approx \mathcal{O}(n^{2.807})$。进一步的计算发现，Strassen 算法所需的加法次数也不超过 $\mathcal{O}(n^{2.807})$。

Strassen 算法在文献中不断改进，至今为止，文献中最好的结果是需要 $\mathcal{O}(n^{2.372})$ 的浮点运算次数。这个工作量任然太大 (而且还涉及稳定性问题)，不难看到，最快的算法不会少于 $\mathcal{O}(n^2)$ 浮点运算次数，但是否存在算法达到这个下届，还是一 可以再讲讲稳定性的内容 上讲，求解一个一般的线性方程组最少需要多少工作量是一个悬而未决的

稀疏矩阵方法 提高求解线性方程组，甚至是一般数学问题的一种有效途径就是充分利用问题的特殊性. 列如，当一个线性方程组 $Ax = b$, A 的非零元的个数只有 $\mathcal{O}(n)$ 时，我们称这样的 A 为稀疏矩阵, 对应的线性方程组成为稀疏的. 稀疏性在目前的科学与工程实践中大量出现, 比如采用有有限元或者有限差分得到的偏微分方程的离散系统一般都是稀疏系统。利用高斯消去时带宽的变化特点，通过特殊的技巧对未知量进行重新排序以减少矩阵的带宽. 然而, 如何最大可能地减少带宽本身是一个 NP-完全问题。对于一些有限元系统, 我们可以通过对自由度列自由度 (未知量), 得到一个带宽较少的系统, 这将极大的减少高斯消去的计算量。

带宽要不要解释一下?

3 高性能计算：硬件与算法的改进

1937 年，世界上第一台计算机 (阿塔纳索夫-贝瑞计算机，简称 ABC 计算机) 被开始设计，不可编程，仅仅用来求解线性代数方程组，并于 1942 年成功进行了测试。从 20 世纪 60、70 年代开始，计算机技术得到飞速发展，极大地推动生产力的发展，直到现在乃至更远将来的每一天都在革新人类生活、工作和研究的方方面面。一个对计算机技术发展速度的预测是著名的摩尔定律，它由英特尔创始人之一戈登 · 摩尔提出，通用的表达是：集成电路上可容纳的电晶体数目，约每隔 18 个月便增加一倍。也就是说相同价格购买的电脑，性能每隔 18 个月增加一倍。

在很多科学计算领域，算法的发展已经可以匹敌硬件技术发展的 Moore's Law. 例如，在大规模线性系统的求解上，计算机诞生之后的 30 多年间，从高斯消去法到多重网格法，求解线性代数系统的算法效率提升了 1600 多万倍，这与同期硬件速度的提升是一致的。实际上，在 2010 年美国 President's Council of Advisors on Science and Technology 的报告就称"Progress in Algorithms Beats Moore's Law"，其中所举的例子是线性规划问题的求解算法。

4 有限元方法及其离散系统

有限元方法是科学计算中最常用的方法之一. 我们以如下 $H(D)$ diffusion problem 为例,

$$(I + D^*D)u = f \qquad \text{in } \Omega, \tag{4}$$

其中 D 是一阶微分算子, D^* 为 D 的 adjoint operator，在三维中可以选 grad, curl, div. 作为一个简单的例子，当 $D = \text{grad}$(或者 ∇)，方程(4)就是二阶椭圆型方程 $-\Delta u + u = f$。

在方程(4)两边乘以测试函数 v，然后分部积分，可以得到如下的弱形式

$$(Du, Dv) + (u, v) = (f, v) \qquad \forall v \in H(D). \tag{5}$$

加上合适的边界条件

这里的测试函数可以跑遍空间 $H(D)$ 内可能的函数. 在合适的函数空间中, 弱问题(5)的解可以证明等价于(4)的解. 当然, 弱问题(5)仍然是不可计算的, 因为 $H(D)$ 的维数是无穷的. 有限元方法从弱问题(5)出发, 对 $H(D)$ 进行如下有限维逼近：首先, 对空间 Ω 进行满足某种性质的剖分 (记为 $\mathcal{T}_h$), 在每个单元内做某种局部逼近. 需要注意的是, 单元与单元之间的函数需要满足某种连续性, 而这个机制通过"自由度"这个概念加以实现. 我们仍然以最低阶 $H(\text{grad})$ 的有限元空间为例, 即分片线性且整体连续的函数空间. 这里, 单元与单元之间的自由度定义为节点上的函数值. 由于线性函数可以被节点值唯一确定, 这个自由度的定义可以保证把局部函数"粘连"起来后整个函数是连续的.

下面我们介绍基函数 φ_i：它在第 i 个自由度上为 1 而其它自由度上为 0. 例如, 对于分片线性连续函数空间, 每个基函数看起来像一个帐篷（见图1). 有了基函数，任何离散空间的函数可以表示为基函数的线性组合，即 $u_h = \sum c_i\varphi_i$, 将 test function 跑遍每个基函数，我们可以得到线性代数系统 $Ac = b$，其中

$$A_{ij} = (\nabla\varphi_j, \nabla\varphi_i) + (\varphi_j, \varphi_i), \qquad b_i = (f, \varphi_i).$$

A 称为*刚度矩阵*, b_j 称为*负载*. 实际计算中, 刚度矩阵通常通过局部组装后再整体组装.

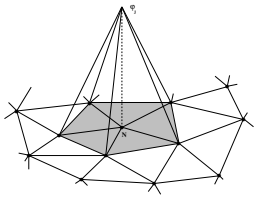


Figure 1: A vertex $N \in \mathcal{V}$ on a mesh and its associated nodal/Lagrange basis φ_j.

经过几十年的发展, 有限元方法已成功应用于电磁学, 材料科学, 天体物理学和成像处理等领域. 这些问题在数学理论和实际应用两个方面都会带来很多挑战. 一个自然的问题：什么样的离散是自然的？一般来说, 一个好的离散化方法应该符合物理和工程问题的底层结构. 这个想法通常在各种情况下被称为“结构保存”. 例如, 在电磁现象的模拟中, 电场自然地在一条路径上被评估（整合）以给出电压的改变; 而对于扩散问题, 在二维微元件上评估体积上的质量变化. 在几何学的语言中, 这些物体被定义为微分 1-形式和 2-形式. 为了在数值计算中模拟这些结构, 很多研究工作考虑微分形式的离散, Finite Element Exterior Calculus.

$$
\begin{tikzcd}
\mathbb{R} \arrow[r] & C^\infty(\Omega) \arrow[r, "\mathrm{grad}"] \arrow[d, "\Pi_h^{\mathrm{grad}}"] & {[C^\infty(\Omega)]^3} \arrow[r, "\mathrm{curl}"] \arrow[d, "\Pi_h^{\mathrm{curl}}"] & {[C^\infty(\Omega)]^3} \arrow[r, "\mathrm{div}"] \arrow[d, "\Pi_h^{\mathrm{div}}"] & C^\infty \arrow[r] \arrow[d, "\Pi_h^{0}"] & 0 \\
\mathbb{R} \arrow[r] & \mathcal{W}_h^0 \arrow[r, "\mathrm{grad}"] & \mathcal{W}_h^1 \arrow[r, "\mathrm{curl}"] & \mathcal{W}_h^2 \arrow[r, "\mathrm{div}"] & \mathcal{W}_h^3 \arrow[r] & 0
\end{tikzcd}
$$

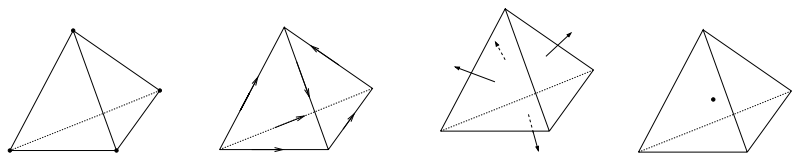


Figure 2: Continuous and discrete de Rham sequences.

具体的说, 我们考虑 $H(\mathrm{curl})$、$H(\mathrm{div})$ 的子空间, 它们分别可以作为电场和磁场的离散空间. 图2给出了最低阶的相应单元，其具体构造如下：

For $H(\mathrm{curl})$ on each element K, the shape function space is 6-dimensional and the degrees of freedoms are tangential integral on each element edge

$$\mathcal{P} = \{\boldsymbol{\alpha} + \boldsymbol{\beta} \times x : \boldsymbol{\alpha}, \boldsymbol{\beta} \in \mathbb{R}^3\}, \quad \mathcal{N} = \left\{ \int_e \boldsymbol{v} \cdot \boldsymbol{t} : \text{for each edge } e \subset K \right\}.$$

Let $e = [a_i, a_j]$ be an edge of the element K and λ_i, λ_j be the barycenter coordinates, then basis function corresponding to the edge e is $\phi_{ij} = \lambda_i \nabla \lambda_j - \lambda_j \nabla \lambda_i$.

For $H(\mathrm{div})$ on each element K is 4-dimensional and the degrees of freedoms are normal integral on each element face

$$\mathcal{P} = \{\boldsymbol{\alpha} + \beta \boldsymbol{x} : \boldsymbol{\alpha} \in \mathbb{R}^3, \beta \in \mathbb{R}\}, \quad \mathcal{N} = \left\{ \iint_F \boldsymbol{v} \cdot \boldsymbol{n} : \text{ for each face } F \subset K \right\}.$$

Let $F = [a_i, a_j, a_k]$ be a face of the element K and $\lambda_i, \lambda_j, \lambda_k$ be the barycenter coordinates, then basis function corresponding to the face F is $\phi_{ijk} = \lambda_i \nabla \lambda_j \times \nabla \lambda_k + \lambda_k \nabla \lambda_i \times \nabla \lambda_j + \lambda_j \nabla \lambda_k \times \nabla \lambda_i$. 通常 $H(\mathrm{curl})$ 和 $H(\mathrm{div})$ 的离散空间分别称为 Nédélec space 和 Raviart-Thomas space, 分别保持离散空间棱上切向连续和面上法向连续. 这些有限元成了一个离散的 de Rham 复型（图2）: 通过微分运算符（grad, curl 和 div）将一个空间映射到另一个空间. 通过这种方式, 可以在三角

剖分上建立一个离散的 de Rham 上同调理论. 拓扑信息忠实地反映在离散化中, 这就是为什么得到的数值算法是稳定的和保持结构的几何解释.

从广义相对论所描述的各种各样的问题, 包括电磁学到弹性, 晶体中的缺陷和错位, 以及时空的涟漪, 微分复型都表现出内在的联系. 这些来自几何和拓扑的见解和工具也激励了许多重要的进展和数值数学和计算. 这些结构保持方法遵循在离散网格和图上发展代数和几何结构的趋势. 例如, 丘成桐教授和合作者开发了计算保形几何和几何图形分析. 这些扎根于现代几何的进展在工程应用中取得了巨大的成功. 几何, 物理, 工程和计算之间的相互作用正在发挥越来越重要的作用，而离散系统的快速求解器是一个重要的研究方向.

After discretization, we get a discretized to solve. Hence in general, we consider how to solve the system

$$Au = f, \tag{6}$$

where $A \in \mathbb{R}^{n\times n}, f \in \mathbb{R}^n$.

与直接法不同，迭代法采取逐次逼近的方法来求解方程组。即从某一个初始值出发，按照一定的计算格式（迭代公式），构造一个向量的无穷序列，其极限才是方程的精确解。A basic iterative method can be written as follows: given $u^0 \in \mathbb{R}^n$,

$$u^m = u^{m-1} + B(f - Au^{m-1}), \quad m = 1, 2, \cdots . \tag{7}$$

where $B \in \mathbb{R}^{n\times n}$ is nonsingular and can be viewed as an approximate inverse of A. Let $\bar{B} = B' + B - B'AB$, then we have the following identity

B bar, B prime 需要说明一下

$$\|I - BA\|_A^2 = 1 - \Big(\sup_{\|v\|_A=1} (\bar{B}^{-1}v, v)\Big)^{-1}. \tag{8}$$

The classical iterative methods such as Jacobi and Gauss-Seidel methods can be viewed as special cases of the iterative method (7). Let us split $A = D + L + U$, where D is is the diagonal of A and L and U are the strictly lower and upper triangular parts of A, respectively. Then the Jacobi and Gauss-Seidel methods can written in the following form

$$u^m = u^{m-1} + B(f - Au^{m-1}), \quad m = 1, 2, \ldots \tag{9}$$

with

$$B = \begin{cases} D^{-1} & \text{Jacobi;} \\ (D+L)^{-1} & \text{Gauss-Seidel.} \end{cases} \tag{10}$$

传统迭代法求解有限元离散的代数系统是很费时的。例如，采用 Gauss-Seidel 求解 Poisson 问题的误差随着迭代部数变化如图3a所示。可以看出，在开始的几步中，误差下降迅速，然后误差下降迅速变得平缓。事实上，using identity (8) , 可以证明 the convergence rate of Gauss-Seidel method is as follows

$$\|I - BA\|_A^2 = 1 - \frac{1}{1 + c_0}. \tag{11}$$

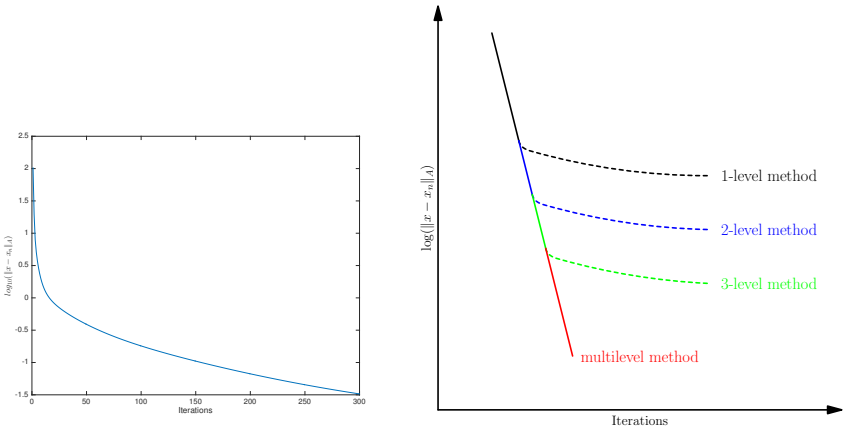
where

$$c_0 = \sup_{v\neq 0} \frac{((LD^{-1}L^T)v, v)}{(v, v)_A}. \tag{12}$$

因此对 Possion 问题离散系统的 convergence rate 为

$$\|\nabla(u_h - u_h^k)\|_0 \le (1 - cn^{-1})^{k/2} \|\nabla(u_h - u_h^0)\|.$$

也就是说，随着问题规模的增加，收敛速度 $\delta = 1 - cn^{-1}$ 会趋近于 1，收敛会越来越慢。



(a) Gauss-Seidel (b) Multigrid

Figure 3: Gauss-Seidel 与 multigrid 的收敛比较

5 Iterative methods and a simple 3×3 system

与直接法不同，迭代法采取逐次逼近的方法来求解方程组。即从某一个初始值出发，按照一定的计算格式（迭代公式），构造一个向量的无穷序列，其极限才是方程的精确解。1940 年,Southwell 提出的松弛迭代法就是一种迭代法 (Southwell, 1940)。在二十世纪前期，迭代法得到了重要的发展，但是系统地研究求解大型线性方程组的迭代法仅始于数字电子计算机发展之后。从二十世纪五十年代开始，我们可以把迭代法的发展分成两个主要时期。第一个时期是以定常迭代法为主的时期，大致是从上世纪四十年代到上世纪七十年代前期。例如雅可比方法 (Jacobi)、高斯－赛德尔方法（Gauss-Seidel）、逐次超松弛（SOR）方法以及这些方法的改进与加速形式。这些迭代法实现简单，但是只能对于一些特殊的矩阵保证收敛。随着科学技术的发展，需要求解的线性方程组的规模越来越大，矩阵的类型越来越广，此时这些方法的收敛速度一般太慢，甚至不收敛。其中最有效的 SOR 方法依赖于参数的选取，但一般无法得到最优参数，从而限制了该方法。

第二个时期是以 Krylov 子空间迭代法结合高效预条件算法为代表的，这个时期开始于上世纪七十年代中期延续至今。子空间方法具有存储量小，计算量小且易于并行等优点，非常适合于并行求解大型稀疏线性方程组。Krylov 子空间迭代法的工作原理是基于初始残量生成一个 Krylov 子空间，每一步在子空间中求一个极小化问题得到一个精确解的近似。例如求解对称正定矩阵的共轭梯度法（CG），求解对称不定矩阵的最小残量法 (MINRES) 和求解一般非对称矩阵的广义最小残量法（GMRES）。因为这些方法都是在一组正交的空间中求解，理论上可以保证 Krylov 子空间迭代法在步迭代以内收敛到精确解。然而在实际中因为舍入误差的存在，我们作使用的 Krylov 子空间的基是不正交的，我们通常用 Krylov 子空间方法作为迭代法。在较大的时候，迭代会在很少的步数就可以达到足够的精度。更重要的是 Krylov 子空间迭代法可以使用一些预条件方法进一步加速其收敛过程。

我们考虑下面这个简单的例子: $A_\epsilon x = b$, 其中 $A_\epsilon = A_0 + \epsilon I$

$$A_0 = \begin{pmatrix} 1 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 1 \end{pmatrix}, \quad b = \begin{pmatrix} -1 \\ -1 \\ 2 \end{pmatrix} \in \mathrm{Range}(A_0).$$

通过简单计算, 可以获得 A_0 的三个特征值 $3, 1, 0$, 其核空间由向量 $e = (1, 1, 1)^T$ 张成. 可以看出, 随着 ϵ 的减小, 矩阵 A_ϵ 逐渐变得奇异.

采用 Gauss-Seidel method 求解如上 3×3 线性系统, 则迭代格式 ($x^{k-1} \to x^k$) 如下:

$$\begin{aligned} a_{11}x_1^k + a_{12}x_2^{k-1} + a_{13}x_3^{k-1} &= b_1, \\ a_{21}x_1^k + a_{22}x_2^k \quad + a_{23}x_3^{k-1} &= b_2, \\ a_{31}x_1^k + a_{32}x_2^k \quad + a_{33}x_3^k \quad &= b_3. \end{aligned}$$

Gauss-Seidel 方法对于 SPD 矩阵一定收敛. 但是, 相应的收敛速度是一个值得研究的问题. 我们可以发现迭代步数随着 ϵ 减小而急剧增长, 也就是说迭代法关于参数 ϵ 是不一致的. 令人惊讶的是, 当 ϵ 变到 0 的一霎那, 迭代步数瞬间从上万变成了 2! (参见表1) 事实上, 通过严格的分析, 当 $\epsilon > 0$ 时, Gauss-Seidel method 的收敛速度满足 $\delta_\epsilon \geq \sqrt{\frac{1}{1+4\epsilon}}$; 而 $\epsilon = 0$ 时则变为 $\delta_0 \leq \frac{1}{\sqrt{2}}$.

如何修正 Gauss-Seidel method 的这个奇怪的现象呢? 将近似奇异向量 e 显式引入到空间中, 于是 $x \in \mathbb{R}^3 = x_1e_1 + x_2e_2 + x_3e_3$ 可以 (不唯一的) 表示为 $x = \underset{\sim}{x}_1e_1 + \underset{\sim}{x}_2e_2 + \underset{\sim}{x}_3e_3 + \underset{\sim}{x}_4e = P\underset{\sim}{x}$, 其中扩充矩阵 $P : \mathbb{R}^3 \mapsto \mathbb{R}^4$ 为

$$P = (e_1, e_2, e_3, e) = \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{pmatrix}.$$

于是我们可以将 $A_\epsilon x = b$ 扩展为如下 4×4 的半正定系统

$$\underset{\sim}{A}_\epsilon \underset{\sim}{x} = \underset{\sim}{b}, \tag{13}$$

其中

$$\underset{\sim}{A}_\epsilon = P^T A_\epsilon P = \begin{pmatrix} A_\epsilon & A_\epsilon e \\ e^T A_\epsilon & e^T A_\epsilon e \end{pmatrix} = \begin{pmatrix} 1+\epsilon & -1 & 0 & \epsilon \\ -1 & 2+\epsilon & -1 & \epsilon \\ 0 & -1 & 1+\epsilon & \epsilon \\ \epsilon & \epsilon & \epsilon & 3\epsilon \end{pmatrix}, \ \underset{\sim}{b} = P^T b = \begin{pmatrix} b \\ e^T b \end{pmatrix} = \begin{pmatrix} -1 \\ -1 \\ 2 \\ 0 \end{pmatrix}.$$

对于扩充系统采用传统的迭代法等价于对原始系统进行多层迭代法. 例如, 对(13)的 Gauss-Seidel method 就是对原系统的两网格 slash-cycle. 表(1)中可以看出对于扩充系统的 Gauss-Seidel method 关于 ϵ 是一致的.

是不是需要更多说明?

# GS for orignal and expanded		
ϵ	original	expanded
1.	18	12
10^{-1}	100	10
10^{-2}	852	6
10^{-3}	6,982	6
10^{-4}	54,470	4
$0._{[\text{singular}]}$	2	–

Table 1: Gauss-Seidel method to the original system and expanded systems with $\|A_\epsilon x^k - b\| \leq 10^{-8}$

6 Conjugate Gradient Method

Conjugate gradient method is one of the most important ten algorithms in last one hundred years. Let us consider to solve $Au = f$ again, where $A \in \mathbb{R}^{n\times n}$ is SPD, $f \in \mathbb{R}^n$. The method to be discussed fall into the category of line-search methods that are based on a sequence of search directions $p_k \in \mathbb{R}^n$ $(k = 0, 1, \ldots)$: at each iteration u_k, we update

$$u_{k+1} = u_k + \alpha_k p_k \quad \text{where} \quad \alpha_k = \arg\min_{\alpha\in\mathbb{R}^1} \|u - (u_k + \alpha p_k)\|_A$$

The question is how to choose the search direction p_k at each iteration. The main idea of conjugate gradient is to choose search direction p_k that are mutually orthogonal with respect to A-inner product, namely

$$(p_i, p_j)_A = \langle Ap_i, p_j\rangle = 0, \quad i \neq j, \tag{14}$$

such that

$$\|u-u_{k+1}\|_A = \min_{\alpha_k\in\mathbb{R}^1} \|u-(u_k+\alpha_k p_k)\|_A = \min_{\alpha_1,\cdots,\alpha_k\in\mathbb{R}^1} \|u-(u_k+\sum_{i=1}^{k}\alpha_i p_i)\|_A.$$

Hence we obtain the following conjugate gradient method

Conjugate gradient method

Given $u_0 \in \mathbb{R}^n$; $r_0 = f - Au_0$; $p_0 = r_0$;
for $k = 0, 1, \ldots,$ till $\|r_k\| \le tol$
  $\alpha_k = \langle r_k, r_k\rangle / \langle Ap_k, p_k\rangle$;
  $u_{k+1} = u_k + \alpha_k p_k$;
  $r_{k+1} = r_k - \alpha_k A p_k$;
  $\beta_k = \langle r_{k+1}, r_{k+1}\rangle / \langle r_k, r_k\rangle$;
  $p_{k+1} = r_{k+1} + \beta_k p_k$.
endfor

Note that, in each iteration, only one evaluation of the action of A and two vector inner products are needed in this algorithm.

The conjugate gradient method can theoretically be viewed as a direct method, as it produces the exact solution after a finite number of iterations, which is not larger than the size of the matrix, in the absence of round-off error. However, the conjugate gradient method is unstable with respect to even small perturbations, e.g., most directions are not in practice conjugate, and the exact solution is never obtained. Fortunately, the conjugate gradient method can be used as an iterative method as it provides monotonically improving approximations u_k to the exact solution u, which may reach the required tolerance after a relatively small (compared to the problem size) number of iterations. In fact, the convergence rate can be given by the following estimate.

$$\|u-u_k\|_A \le 2\left(\frac{\sqrt{\kappa(A)}-1}{\sqrt{\kappa(A)}+1}\right)^k \|u-u_0\|_A,$$

where $\kappa(A) = \frac{\lambda_{\max}(A)}{\lambda_{\min}(A)}$ denotes the condition number of A.

From the convergence result of CG method, we can see that if $\kappa(A)$ is very big, the convergence of CG method will be slow. Hence we can construct a preconditioner B for A such that $\kappa(BA) \ll \kappa(A)$. The preconditioner should be SPD and easy to achieve. Then solving $Au = f$ is equivalent to solving $BAu = Bf$. And the CG method for solving $BAu = Bf$ can be much faster.

7 多层迭代法的一般框架

一般来说，线性系统的解由不同频率的解线性组合而成。基于这个性质，多层迭代法的基本思想是对每种频率的设计相应的求解方式。如何对单一的线性空间刻画出不同频率的解呢？我们考虑 V 的扩展空间 $\underset{\sim}{V} = V_0 \times V_2 \cdots V_J$，并且定义 $\Pi = (\Pi_0, \Pi_1, \cdots, \Pi_J) : \underset{\sim}{V} \mapsto V$。一个 V 中的函数可以表示为不同频率解的和，即 $u = \sum_{i=0}^{J} \Pi_i u_i = \Pi \underset{\sim}{u}$。于是，线性系统 $Au = f$ 可以写成如下扩展系统

$$\underset{\sim}{A}\underset{\sim}{u} = \Pi' A \Pi \underset{\sim}{u} = \Pi' f = \underset{\sim}{f}.$$

其中，$\underset{\sim}{A} = \Pi' A \Pi$, 即线性算子 A 扩展为

$$\underset{\sim}{A} := \begin{pmatrix} \Pi_0' A \Pi_0 & \Pi_0' A \Pi_1 & \cdots & \Pi_0' A \Pi_J \\ \Pi_1' A \Pi_0 & \Pi_1' A \Pi_1 & \cdots & \Pi_1' A \Pi_J \\ \vdots & \vdots & \ddots & \vdots \\ \Pi_J' A \Pi_0 & \Pi_J' A \Pi_1 & \cdots & \Pi_J' A \Pi_J \end{pmatrix} : \underset{\sim}{V} \mapsto \underset{\sim}{V}. \tag{15}$$

需要注意对称正定系统对应的扩展系统是半正定的。可以看出，扩展系统包含了不同的频率空间，因此，可以直接对 $\underset{\sim}{A}\underset{\sim}{u} = \underset{\sim}{f}$ 进行标准的迭代法，从而得到对于 $Au = f$ 的多层迭代法。应用两种标准的迭代法 (parallel subspace correction 和 successive subspace correction) 于扩展系统，我们得到相应的算法：

Algorithm 1 Parallel subspace correction

1: **for** $i = 0 : J$ **do**
2: $\quad e_i = R_i \Pi_i'(f - Au^{n-1})$.
3: $u^n = u^{n-1} + \sum_{i=0}^{J} \Pi_i e_i$.

Algorithm 2 Successive subspace correction

1: $u_{-1}^{n-1} = u^{n-1}$
2: **for** $i = 0 : J$ **do**
3: $\quad e_i = R_i \Pi_i'(f - Au_{i-1}^{n-1})$.
4: $\quad u_i^{n-1} = u_{i-1}^{n-1} + \Pi_i e_i$.
5: $u^n = u_J^{n-1}$.

一般来说，PSC 对应的 smoother 常被用做预条件子，需要结合其它方法（比如上文提到的共轭梯度法）来迭代求解；而 SSC 常被用来作为迭代法直接求解线性系统。下面我们介绍两种重要的预条件子：Bramble-Pasciak-Xu 和 Hiptmair-Xu 预条件子，它们都可以看作对扩展系统进行块对角预条件，即 $\underset{\sim}{B} = \underset{\sim}{R}$ and $B = \Pi_i \underset{\sim}{B} \Pi_i'$ 或者 $B = \sum_{i=0}^{J} \Pi_i R_i \Pi_i'$,. Further,

$$\langle B^{-1} v, v \rangle = \inf_{\Pi \underset{\sim}{v} = v} \langle \underset{\sim}{B}^{-1} \underset{\sim}{v}, \underset{\sim}{v} \rangle.$$

在介绍两种预条件子之前，我们先给出下面的 Fast Auxiliary Space Preconditioning Theorem，作为设计和分析预条件子的一般原则。

Theorem 1 *如果 $\|\cdot\|_A, \|\cdot\|_{\underset{\sim}{B}^{-1}}$ 和 Π 满足如下条件：*

1.（稳定分解）对于任意 $v \in V$，存在 $\underset{\sim}{v} \in \underset{\sim}{V}$，使得 $\Pi \underset{\sim}{v} = v$ 并且 $\|\underset{\sim}{v}\|_{\underset{\sim}{B}^{-1}}^2 \le c_0 \|v\|_A^2$；

2.（Π 有界性）对于任意 $\underset{\sim}{v} \in \underset{\sim}{V}$，$\|\Pi \underset{\sim}{v}\|_A^2 \le c_1^{-1} \|\underset{\sim}{v}\|_{\underset{\sim}{B}^{-1}}^2$;

那么 $\kappa(BA) \le c_0/c_1$.

8 范数等价与 BPX 预条件子

BPX 预条件子在 1990 年由 Bramble, Pasciak, Xu 提出 [1], 是二阶椭圆型问题的高效简便的预条件子, 现已成为科学计算中最基本的算法之一. 根据 Theorem 1, 构造预条件子的核心为选取合适的扩展空间, 并构造 $\underset{\sim}{B}$ 满足稳定分解. 这里我们首先假设有下面一组嵌套的

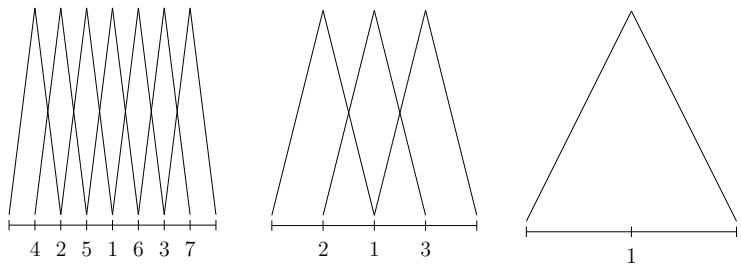


Figure 4: 一维多层基函数

子空间 $V_0 \subset V_1 \subset \cdots V_J = V \subset H_0^1(\Omega)$, 并且相对应的网格 $\mathcal{T}_i$ 是拟一致的, 网格的尺度 $h_i = \gamma^{2i}(0 < \gamma < 1)$.

以一维为例, 不同层的有限元空间由图4所示. 由于 V_i 为 V 的子空间, Π_i 为自然的 inclusion operator. 于是, 扩展系统对应的刚度矩阵为

$$\underset{\sim}{A} = \Big((\varphi_{i,k}, \varphi_{j,l})_A \Big) \in \mathbb{R}^{N\times N}, \qquad N = \sum_{i=0}^{J} n_i.$$

这里 $\varphi_{i,k}$ 为 V_i 上第 k 个基函数. 本节和核心为如下的范数等价:

Theorem 2 (范数等价) 对于任意 $v \in V$,

$$\|v\|_1^2 \eqsim \sum_{i=0}^{J} \|(Q_i - Q_{i-1})v\|_1^2 \eqsim \sum_{k=0}^{J} h_i^{-2} \|(Q_i - Q_{i-1})v\|_0^2,$$

其中 Q_i 为 V_i 上的 L^2 投影。

The above theorem can be written in the following equivalent form:

$$\langle -\Delta v, v\rangle \eqsim \sum_{k=0}^{J} h_i^{-2}((Q_i - Q_{i-1})v, v), \quad \forall v \in H_0^1(\Omega).$$

Thus, in the sense of spectral equivalence

$$-\Delta \eqsim G \equiv \sum_{k=0}^{J} h_i^{-2}((Q_i - Q_{i-1})$$

it is easy to see that

$$G^s \equiv \sum_{k=0}^{J} h_i^{-2s}((Q_i - Q_{i-1}), \quad s \in [-1, 1]$$

In particular,

$$(-\Delta)^{-1} \eqsim G^{-1} \equiv \sum_{k=0}^{J} h_i^2 (Q_i - Q_{i-1})$$

Thus

$$((-\Delta)^{-1} f, f) \eqsim (G^{-1} f, f) \equiv \sum_{k=0}^{J} h_i^2((Q_i - Q_{i-1})f, f) \eqsim \sum_{k=0}^{J} h_i^2 (Q_i f, f) \eqsim \sum_{k=0}^{J} (R_i Q_i f, f),$$

where $R_i = h_i^2 I_i$ is the Richardson method on level i.

9 HX 预条件子

HX 预条件子再 2007 年由 Hiptmair 和 Xu 提出 [2], 是电磁场方程最有效的快速求解方法之一, and it was reported by the U.S. Department of Energy as one of the top 10 breakthroughs in computational science in recent years. 其主要想法是利用离散的 de Rham sequence 和 regular decomposition 来设计(4)对于 $D = \mathrm{curl}, \mathrm{div}$ 的求解器。

我们以 $D = \mathrm{curl}$ 为例, 注意到 $\mathrm{curl}^* \mathrm{curl}$ 算子有很大的核空间, the exactness of de Rham sequence 给出了核空间的精确的描述: range of grad. 因此, 快速迭代法的重点是如何刻画出微分算子的核空间。首先在连续函数空间上 $H(\mathrm{curl})$ 有 regular decomposition$H(\mathrm{curl}) = \boldsymbol{H}^1 + \mathrm{grad}\, H^1$. 在离散层面, 根据图2, Hiptmair and Xu 给出了如下的 discrete regular decomposition

$$\boldsymbol{E}_h = \tilde{\boldsymbol{E}}_h + \Pi_h^1 \boldsymbol{\Psi}_h + \mathrm{grad}\, \varphi_h \tag{16}$$

也就是说任何 Nédélec edge element 函数 $\boldsymbol{E}_h \in \mathcal{W}_h^1$ 可以被分解为一个高频项 $\tilde{\boldsymbol{E}}_h \in \mathcal{W}_h^1$, 一个向量型的 Lagrange elements$\boldsymbol{\Psi}_h \in \mathcal{W}_h^0$, 和一个 gradient of Lagrange element $\varphi_h \in \mathcal{W}_h^0$. 其中, 最后一项就是 curl 在 Nédélec edge element 下的核空间.

根据一般框架, 考虑 $V_h = \mathcal{W}_h^1$ 的扩展空间 $\underset{\sim}{V}_h = \mathcal{W}_h^1 \times \boldsymbol{\mathcal{W}}_h^0 \times \mathcal{W}_h^0$, 而相应的 $\Pi = (\mathrm{id}, \Pi_h^1, \mathrm{grad})$. 由于 $\tilde{\boldsymbol{E}}_h$ 是 $\boldsymbol{E}$ 的高频部分, 我们定义 $\|\cdot\|_{B_0^{-1}} = h^{-1}\|\cdot\|_0$, 而 $\boldsymbol{\mathcal{W}}_h^0$ 和 $\mathcal{W}_h^0$ 上采用其本身定义的 $\|\cdot\|_1$. 这样 Theorem 1中的稳定分解是自然满足的, 因为

$$h^{-1}\|\tilde{\boldsymbol{E}}_h\|_0 \lesssim \|\boldsymbol{E}_h\|_{H(\mathrm{curl})}, \qquad \|\boldsymbol{\Psi}\|_1 \lesssim \|\boldsymbol{E}_h\|_{H(\mathrm{curl})}, \qquad \|\varphi_h\|_1 \lesssim \|\boldsymbol{E}_h\|_{H(\mathrm{curl})}.$$

由 inverse inequality 和 discrete de Rham sequence, Π 的有界性是容易的. 相应地, HX 预条件子可以写为:

$$B^{\mathrm{curl}} = S^{\mathrm{curl}} + \Pi^{\mathrm{curl}} B^{\mathrm{grad}} (\Pi^{\mathrm{curl}})^t + \mathrm{grad}\, B^{\mathrm{grad}} (\mathrm{grad})^t.$$

具体实现中, 高频部分可以由原始系统的 Jacobi 或 Gauss-Seidel method S^{curl} 加以处理, 而 $\boldsymbol{\mathcal{W}}_h^0$ 和 $\mathcal{W}_h^0$ 则需要调用 $H(\mathrm{grad})$ 求解器 B^{grad}, 比如上文中介绍的 BPX 预条件子或者代数多重网格方法 (AMG). 也就是说, 3 维 HX 预条件子一次迭代中需要调用 4 次 Poisson 求解器. 我们在一般框架中给出了 HX 预条件子的有效性. 因此, B^{curl} 是算子 $I + \mathrm{curl}^* \mathrm{curl}$ 的一个好的 (谱等价) 近似逆.

同样的想法可以推广到 $D = \mathrm{div}$ 以及一般的 de Rham 复形. 例如在知道了如何对于 $D = \mathrm{curl}$ 求解近似逆之后, 对于 $D = \mathrm{div}$, 我们有

$$\begin{aligned} B^{\mathrm{div}} &= S^{\mathrm{div}} + \Pi^{\mathrm{div}} B^{\mathrm{curl}} (\Pi^{\mathrm{div}})^t + \mathrm{curl}\, B^{\mathrm{curl}} (\mathrm{curl})^t \\ &= S^{\mathrm{div}} + \Pi^{\mathrm{div}} B^{\mathrm{grad}} (\Pi^{\mathrm{div}})^t + \mathrm{curl}\, S^{\mathrm{curl}} (\mathrm{curl})^t + \mathrm{curl}\, \Pi^{\mathrm{curl}} B^{\mathrm{grad}} (\Pi^{\mathrm{curl}})^t (\mathrm{curl})^t. \end{aligned}$$

此时一次迭代需要调用 6 次 Poisson 求解器.

10 一些应用例子

10.1 麦克斯韦方程

1873 年，James Maxwell 用严谨的数学提出了位移电流的假说，将电磁场基本定律归结为一组偏微分方程，这就是著名的麦克斯韦方程（Maxwell's Equation）。通过对方程组的分析，他预见了电磁波的存在，推动了电磁学的迅猛发展。1887 年 Heinrich Hertz 用实验方法产生和检测到了电磁波，证实了麦克斯韦的预见。电磁场模拟在很多物理和工程领域都有大量的应用，例如半导体芯片研发、隐形飞机设计和发电机制造等。处理大规模电磁模拟问题时，模拟效果受到所用线性代数方程组解法器性能的制约。2008 年，美国能源部发布报告，介绍近年来应用数学和计算机科学领域中出现的十大突破性成就。我与合作者提出的一个求解麦克斯韦方程的预条件方法（HX 算法 [2]），作为其中唯一一个数值算法入选为十大突破性成就之一。该算法的有效性不太受问题规模和处理器负载的增长的影响，计算性能较传统方法有显著改善。通过美国劳伦斯 · 利弗莫尔国家实验室所做的大量的数值实验证实，利用该新算法可以缩短计算时间 4 至 25 倍。新的算法可以处理几何形状比较复杂以及材料系数有大的间断的问题。特别值得指出的是，此方法是第一个有数学理论保证的适用于麦克斯韦方程的可扩展算法，其工作原理的核心是将原问题转化为可以用传统方法个别处理的一组方程。

10.2 磁流体方程

磁流体力学（Magnetohydrodynamics 或 MHD）主要研究等离子体和磁场的相互作用，其基本方程就是流体力学中的 Navier-Stokes 方程和电动力学中的 Maxwell 方程，被广泛应用于天体物理、可控核聚变等。磁流体力学关心的是流体元的平均效果，它将等离子体作为连续介质处理，要求其特征尺度远大于粒子的平均自由程、特征时间远大于粒子的平均碰撞时间。磁流体力学是由 Hannes Alfvén 建立的，因此成就他于 1970 年获得诺贝尔物理学奖。由于 MHD 方程的复杂性，其数值模拟一直以来都是一个挑战。选择不同的离散方法可能会给出性质完全不同的代数方程组，需要选择与之相适应的迭代求解算法。事实上，选择一个稳定的保结构的离散方法对构造离散代数求解器也起到至关重要的作用。我与合作者提出了一种数值解自然地保持高斯定律的稳定有限元方法 [3, 4]，针对该离散方法所得到线性代数方程组，我们也设计了快速稳健的预条件方法 [7]，大大提高了计算效率。

10.3 流固耦合模拟

在很多实际问题中，多个物理场 (如温度场、应力场、速度场等) 叠加在一起，这些物理场之间的相互作用给理论研究、实验以及数值模拟带来了巨大的挑战。多物理场耦合 (multi-physics) 问题就是要分析多个物理场之间的交互作用，例如: 流固耦合分析、热应力问题、压电分析等等。多物理耦合问题的数值模拟不仅需要求解多个不同物理场 (不同模型、不同尺度、不同特性) 的问题，还需要解决它们之间的非线性耦合。设计适合多物理耦合问题的高效数值求解方法、高效地使用并行计算机进行大规模数值模拟、提高算法的可扩展性和计算效率等是求解复杂耦合问题的重点。流固耦合问题 (Fluid-Structure Interaction 或 FSI) 是目前多物理场、多学科交叉领域研究的热点问题之一。流固耦合问题是一类典型的高性能科学计算问题，其应用涵盖了许多重要的高性能计算领域，其数值模拟亦涉及到大　的共性计算方法，如网格变形、多网格匹配、界面问题、非线性耦合求解等。同时，实际应用中的流固耦合计算问题通常网格规模和计算规模都非常大，必须借助高性能计算机采取并行计算方法来解决，[9, 10, 11], 因而必须研究相应的高效并行算法和高效并行实现技术。

10.4 油藏数值模拟

石油与天然气是一次性的开采资源，据美国能源署 (DOE) 的数据显示，目前世界油田平均采收率为 35% 左右，而一些生产优化技术能把采收率提高到 40%–45%。通过数值模拟手段虚拟实施各种开发方案是定量地评价开发方案和提高油气采收率的重要方法，也是优化油气资源资产价值最有效的工具之一。油藏数值模拟就是利用计算机来求解描述油藏中渗流过程的数学模型，

从而得到能够反映油藏动态的数值解，也就是得到压力、饱和度、温度和产量等关于时间和空间的函数。油藏数值模拟技术起步于上世纪 50 年代，到 70 年代后期该技术逐渐成熟，并出现了以黑油模型、组分模型、混相驱模型、热采模型为基础的等各种模拟器。目前，油藏数值模拟技术已从单一的油藏工程工具渗透到整个油气开发过程中，并成为油藏工程师进行油田开发方案设计和油田开发动态分析等工作不可缺少的有力工具。在油藏模拟计算中 (特别是在用全隐式方法求解大规模问题的)，求解代数线性方程组时候所占了整个计算时间的比例常超过 80%；而且，随着油藏模型的复杂化和精细化，离散线性代数方程组的规模增大、性态变坏，越来越难求解，这个比例还会变得越来越高。所以，提高线性代数方程组的求解速度是缩短数值模拟时间最有效的途径之一，它直接决定了油藏模拟器的求解能力和速度。油藏数值模拟中的大规模线性方程组的系数矩阵常常不具对称正定性，经典迭代法的收敛性得不到保证。利用油藏渗流过程的物理性质、偏微分方程的特性以及离散方法的代数性质，在辅助空间预优方法的理论框架下，我与合作者给出一系列针对全隐式多阶段预条件算法和一些针对半隐式方法的预条件算法 [5, 8, 6]，在中石油的油藏数值模拟软件中得到应用，取得了很好的实际效果。

References

[1] James H Bramble, Joseph E Pasciak, and Jinchao Xu. Parallel multilevel preconditioners. *Mathematics of Computation*, 55(191):1–22, 1990.

[2] Ralf Hiptmair and Jinchao Xu. Nodal auxiliary space preconditioning in $H(\mathrm{curl})$ and $H(\mathrm{div})$ spaces. *SIAM Journal on Numerical Analysis*, 45(6):2483–2509, 2007.

[3] Kaibo Hu, Yicong Ma, and Jinchao Xu. Stable finite element methods preserving \nabla\cdot\varvec {B}= 0 · b= 0 exactly for mhd models. *Numerische Mathematik*, 135(2):371–396, 2017.

[4] Kaibo Hu and Jinchao Xu. Structure-preserving finite element methods for stationary mhd models. *arXiv preprint arXiv:1503.06160*, 2015.

[5] Zheng Li, Shuhong Wu, Jinchao Xu, and Chensong Zhang. Toward cost-effective reservoir simulation solvers on gpus. *Advances in Applied Mathematics and Mechanics*, 8(6):971–991, 2016.

[6] Zheng Li, Shuhong Wu, Chen-Song Zhang, Jinchao Xu, Chunsheng Feng, and Xiaozhe Hu. Numerical studies of a class of linear solvers for fine-scale petroleum reservoir simulation. *Computing and Visualization in Science*, 18(2-3):93–102, 2017.

[7] Yicong Ma, Kaibo Hu, Xiaozhe Hu, and Jinchao Xu. Robust preconditioners for incompressible mhd models. *Journal of Computational Physics*, 316:721–746, 2016.

[8] Changhe Qiao, Shuhong Wu, Jinchao Xu, and Chen-Song Zhang. Analytical decoupling techniques for fully implicit reservoir simulation. *Journal of Computational Physics*, 336:664–681, 2017.

[9] Pengtao Sun, Jinchao Xu, and Lixiang Zhang. Full eulerian finite element method of a phase field model for fluid–structure interaction problem. *Computers & Fluids*, 90:1–8, 2014.

[10] Jinchao Xu and Kai Yang. Well-posedness and robust preconditioners for discretized fluid–structure interaction systems. *Computer Methods in Applied Mechanics and Engineering*, 292:69–91, 2015.

[11] Kai Yang, Pengtao Sun, Lu Wang, Jinchao Xu, and Lixiang Zhang. Modeling and simulations for fluid and rotating structure interactions. *Computer Methods in Applied Mechanics and Engineering*, 311:788–814, 2016.